Dan Haniff: Comparing the relationship with URL length and phishing instances. **[PYTHON]**

One of the most surprising outcomes of this analysis was how consistently the data challenged common assumptions about what a phishing URL actually looks like. Most people would assume that phishing links are long, cluttered, and visually suspicious. However, the data showed the opposite. Phishing URLs averaged 46.9 characters compared to 63.0 for legitimate URLs, and the Very Short bin accounted for the largest absolute volume of phishing URLs at 74,070 cases. At the same time, the Pearson correlation of -0.1298 confirms that URL length alone is a weak predictor, and the U-shaped risk pattern visible in the heatmap shows that both extremely short and extremely long URLs carry elevated phishing rates. This suggests that attackers do not follow a single approach. Some keep URLs short and simple to avoid raising red flags, while others use long, complex strings to hide the true destination. A detection system that relies on URL length as a single feature would miss both of these strategies. The bigger takeaway is that phishing detection needs to account for multiple URL characteristics at once, including length, path depth, query parameters, and domain structure, rather than treating any one feature as a definitive signal on its own.

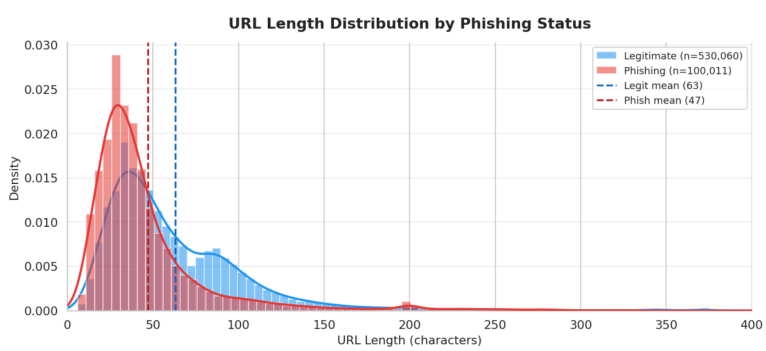
```
=== URL Length Summary Statistics ===
```

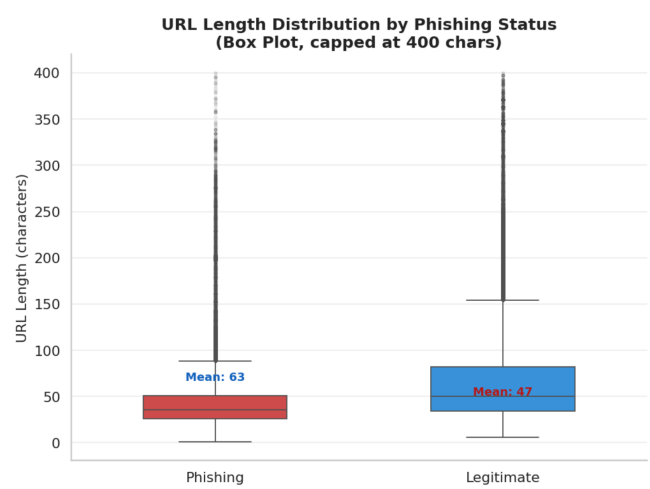
Label	count	mean	std	min	25%	50%	75%	max
Legitimate	530060.0	63.00	45.15	6.0	34.0	50.0	82.0	2175.0
Phishing	100011.0	46.92	43.48	1.0	26.0	36.0	52.0	1779.0

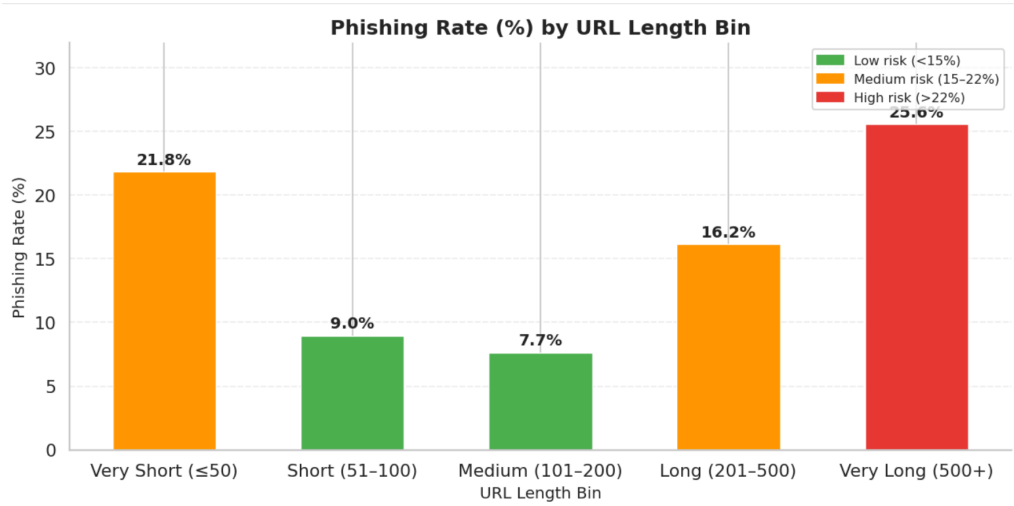
```
Mean URL Length  -->  Legitimate: 63.0 chars  |  Phishing: 46.9 chars
Median URL Length -->  Legitimate: 50 chars  |  Phishing: 36 chars
Pearson Correlation (UrlLength <--> Phising): -0.1298
```

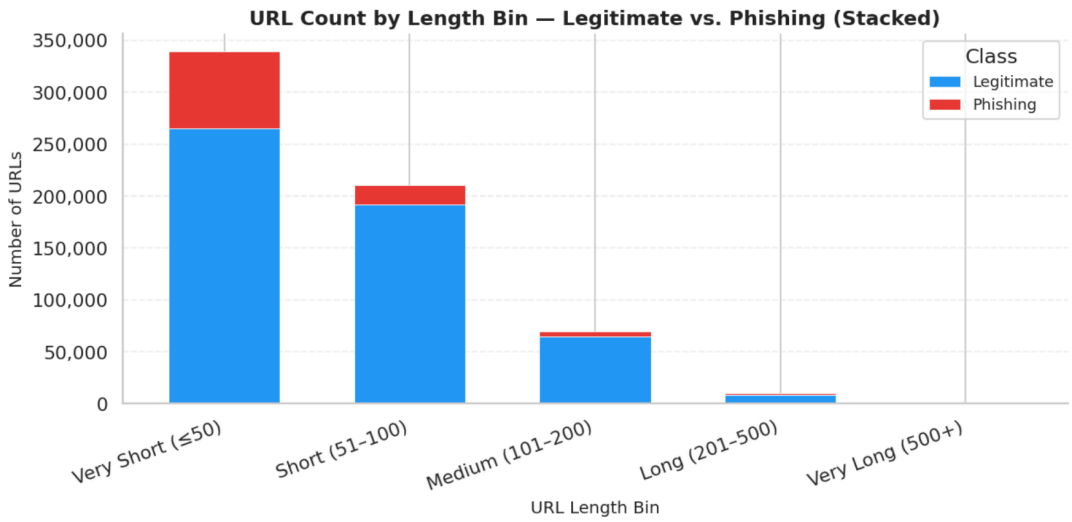
```
=== Phishing Rate by URL Length Bin ===
```

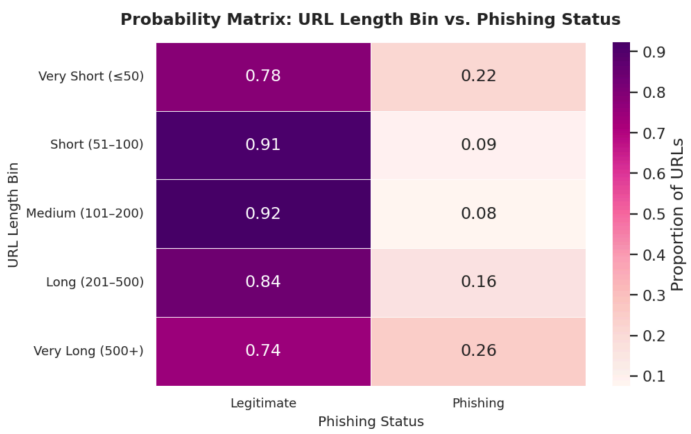
LengthBin	Total	Phishing	Legitimate	Phishing_Rate_%
Very Short (≤50)	339236	74070	265166	21.83
Short (51–100)	210380	18882	191498	8.98
Medium (101–200)	69991	5356	64635	7.65
Long (201–500)	10331	1669	8662	16.16
Very Long (500+)	133	34	99	25.56












```
============================================================
        URL LENGTH vs. PHISHING — KEY FINDINGS
============================================================
  Total URLs analyzed     : 630,071
  Legitimate              : 530,060 (84.1%)
  Phishing                : 100,011 (15.9%)

  Avg length — Legitimate : 63.0 chars
  Avg length — Phishing   : 46.9 chars
  Difference              : 16.1 chars (legit URLs are LONGER)

  Pearson correlation     : -0.1298 (negative = longer -> less phishing)
  Highest-risk bin        : Very Long (500+) -> 25.6% phishing rate

  INTERPRETATION:
  Contrary to common assumptions, phishing URLs in this
  dataset tend to be SHORTER than legitimate ones. Very
  short URLs (≤50 chars) have a ~22% phishing rate, and
  very long URLs (500+) spike again at ~26%. The safest
  range is 51–200 characters (~8–9% phishing rate).
============================================================
```

Statistical Analysis of URL Length and Phishing

To move beyond descriptive observations, several formal statistical tests were conducted to evaluate the relationship between URL length and phishing status across the 630,071 URLs in this dataset.

A Welch's t-test was used to compare the mean URL length between legitimate and phishing URLs, accounting for the unequal variances between the two groups. The results were statistically significant ($t = 106.58$, $p < 0.001$), confirming that the difference in average URL length between legitimate URLs (mean = 63.0 characters) and phishing URLs (mean = 46.9 characters) is not due to chance. To assess the practical significance of this difference, Cohen's d was calculated at 0.36, which falls in the small-to-medium range. This means that while the difference is real and statistically reliable, URL length alone explains only a modest portion of the variation between classes, which is an important distinction to make given the large sample size.

Since URL length is not normally distributed in either class, a non-parametric Mann-Whitney U test was also conducted as a robustness check. The result ($U = 34{,}978{,}147{,}304$, $p < 0.001$) confirmed that the two distributions are significantly different from one another, reinforcing the t-test findings without relying on normality assumptions.

A point-biserial correlation was computed between URL length and the binary phishing label, yielding $r = -0.1298$ ($p < 0.001$). The negative direction confirms that longer URLs are associated with a lower probability of being phishing in this dataset. However, the magnitude of the correlation is weak, which is consistent with the Cohen's d result. Statistical significance here is driven largely by the sample size of over 600,000 records, so the effect size is the more meaningful figure for practical interpretation.

To examine whether phishing rates differ significantly across URL length categories, a chi-square test of independence was performed on a 5x2 contingency table comparing URL length bins against phishing status. The result was highly significant ($X2(4) = 20{,}076.22$, $p < 0.001$), confirming that phishing rates are not evenly distributed across length categories. Cramer's V, which measures the strength of association, was calculated at 0.18, indicating a weak but consistent relationship. The conditional phishing rates per bin tell the more interesting story: URLs in the Short (8.98%) and Medium (7.65%) bins sit well below the overall dataset phishing rate of 15.87%, while the Very Short (21.83%) and Very Long (25.56%) bins exceed it considerably.

Across all four tests, the relationship between URL length and phishing status is statistically significant but modest in practical effect size. The data does not support using URL

length as a standalone classifier. What it does support is using URL length as one feature within a broader detection framework, particularly flagging URLs below 50 characters and above 500 characters as warranting additional scrutiny. The convergence of parametric and non-parametric results adds confidence that these findings are robust and not artifacts of distributional assumptions.

References

Anthropic. (2025). *Claude* (claude-sonnet-4-6) [Large language model]. https://www.anthropic.com

Anti-Phishing Working Group. (2025). *Phishing activity trends report, Q4 2025*. https://apwg.org/trendsreports/

Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications, 25*(2), 443–458. https://doi.org/10.1007/s00521-013-1490-z

Code Cells [PYTHON]:

Import:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.ticker as mticker
import seaborn as sns

# Format:
sns.set_theme(style="whitegrid", font_scale=1.1)
plt.rcParams.update({
    "figure.dpi": 130,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.family": "DejaVu Sans"
})
```

Import phishing.csv:

```python
# importing phishing.csv

df = pd.read_csv("phishing.csv", index_col=0)

# Drop Rows
df = df.dropna(subset=["Phising"])
df["Phising"] = df["Phising"].astype(int)

# Columns
df["Label"] = df["Phising"].map({0: "Legitimate", 1: "Phishing"})

print(f"Dataset loaded: {len(df):,} rows × {df.shape[1]} columns")
print(f" Legitimate URLs : {(df.Phising == 0).sum():,}")
print(f" Phishing URLs   : {(df.Phising == 1).sum():,}\n")
df.head()
```

```
Dataset loaded: 630,071 rows × 13 columns
 Legitimate URLs : 530,060
 Phishing URLs   : 100,011
```

Descriptive Statistics:

```python
# Step 3: Descriptive Statistics:
summary = df.groupby("Label")["UrlLength"].describe().round(2)
print("=== URL Length Summary Statistics ===")
display(summary)

legit = df[df["Phising"] == 0]["UrlLength"]
phish = df[df["Phising"] == 1]["UrlLength"]

print(f"\nMean URL Length  -->  Legitimate: {legit.mean():.1f} chars  |  Phishing: {phish.mean():.1f} chars")
print(f"Median URL Length -->  Legitimate: {legit.median():.0f} chars  |  Phishing: {phish.median():.0f} chars")
corr = df["UrlLength"].corr(df["Phising"])
print(f"Pearson Correlation (UrlLength <--> Phising): {corr:.4f}")
```

Phishing Rate by URL:

```
# Step 4: Bin URLs by length and compute phishing rate
bins   = [0, 50, 100, 200, 500, df["UrlLength"].max() + 1]
labels = ["Very Short (≤50)", "Short (51–100)", "Medium (101–200)",
          "Long (201–500)", "Very Long (500+)"]

df["LengthBin"] = pd.cut(df["UrlLength"], bins=bins, labels=labels, right=True)

bin_stats = df.groupby("LengthBin", observed=True).agg(
    Total    = ("Phising", "count"),
    Phishing = ("Phising", "sum")
)
bin_stats["Legitimate"]      = bin_stats["Total"] - bin_stats["Phishing"]
bin_stats["Phishing_Rate_%"] = (bin_stats["Phishing"] / bin_stats["Total"] * 100).round(2)

print("=== Phishing Rate by URL Length Bin ===")
display(bin_stats)
```

Histogram:

```
# Plot 1: Overlapping Histogram with Kernel Density Estimation (KDE) - shows how URL lengths are distributed for each class.
# Phishing URLs are clustering at shorter lengths
CAP = 400  # cap x-axis for readability (outliers beyond 400 are rare)

fig, ax = plt.subplots(figsize=(11, 5))

for label, color, zorder in [("Legitimate", "#2196F3", 1), ("Phishing", "#E53935", 2)]:
    data = df[df["Label"] == label]["UrlLength"]
    data_capped = data[data <= CAP]
    ax.hist(data_capped, bins=80, density=True, alpha=0.55,
            color=color, label=f"{label} (n={len(data):,})", zorder=zorder)
    # Overlay KDE
    from scipy.stats import gaussian_kde
    kde = gaussian_kde(data_capped, bw_method=0.15)
    x_range = np.linspace(0, CAP, 500)
    ax.plot(x_range, kde(x_range), color=color, linewidth=2.2, zorder=zorder+1)

# Mean lines
ax.axvline(legit.mean(), color="#1565C0", linestyle="--", linewidth=1.8, label=f"Legit mean ({legit.mean():.0f})")
ax.axvline(phish.mean(), color="#B71C1C", linestyle="--", linewidth=1.8, label=f"Phish mean ({phish.mean():.0f})")

ax.set_title("URL Length Distribution by Phishing Status", fontsize=15, fontweight="bold", pad=14)
ax.set_xlabel("URL Length (characters)", fontsize=12)
ax.set_ylabel("Density", fontsize=12)
ax.set_xlim(0, CAP)
ax.legend(fontsize=10, framealpha=0.9)
ax.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.savefig("plot1_distribution.png", dpi=150, bbox_inches="tight")
plt.show()
```

Box Plot:

```
# Plot 2: Box Plot - Median, Spread, and Outliers
fig, ax = plt.subplots(figsize=(8, 6))

df_cap = df[df["UrlLength"] <= 400].copy()
sns.boxplot(data=df_cap, x="Label", y="UrlLength",
            hue="Label", palette={"Legitimate": "#2196F3", "Phishing": "#E53935"},
            width=0.5, fliersize=2, flierprops=dict(alpha=0.15),
            legend=False, ax=ax)

# Annotate means
for i, (label, color) in enumerate([("Legitimate", "#1565C0"), ("Phishing", "#B71C1C")]):
    m = df[df["Label"] == label]["UrlLength"].mean()
    ax.text(i, m + 4, f"Mean: {m:.0f}", ha="center", fontsize=10,
            fontweight="bold", color=color)

ax.set_title("URL Length Distribution by Phishing Status\n(Box Plot, capped at 400 chars)",
             fontsize=14, fontweight="bold")
ax.set_xlabel("", fontsize=12)
ax.set_ylabel("URL Length (characters)", fontsize=12)
ax.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.savefig("plot2_boxplot.png", dpi=150, bbox_inches="tight")
plt.show()
```

Heat Map:

```
# Plot 3: Probability Heatmap Matrix
pivot = df.groupby(["LengthBin", "Label"], observed=True).size().unstack(fill_value=0)
pivot_pct = pivot.div(pivot.sum(axis=1), axis=0).round(4)

fig, ax = plt.subplots(figsize=(8, 5))
sns.heatmap(pivot_pct, annot=True, fmt=".2f", cmap="RdPu",
            linewidths=0.5, linecolor="white",
            cbar_kws={"label": "Proportion of URLs"},
            ax=ax)

ax.set_title("Probability Matrix: URL Length Bin vs. Phishing Status",
             fontsize=13, fontweight="bold", pad=14)
ax.set_xlabel("Phishing Status", fontsize=11)
ax.set_ylabel("URL Length Bin", fontsize=11)
ax.set_xticklabels(["Legitimate", "Phishing"], fontsize=10)
ax.set_yticklabels(ax.get_yticklabels(), rotation=0, fontsize=10)

plt.tight_layout()
plt.savefig("plot3_heatmap.png", dpi=150, bbox_inches="tight")
plt.show()
```

Bar Chart:

```
# Plot 4: Phishing Rate Bar Chart — Phishing Rate by URL Length Bin — (Which URL Length ranges are most dangerous?)
rates  = bin_stats["Phishing_Rate_%"]
colors = ["#4CAF50" if r < 15 else "#FF9800" if r < 22 else "#E53935" for r in rates]

fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.bar(bin_stats.index, rates, color=colors, edgecolor="white",
              linewidth=0.8, width=0.55)

# Value labels
for bar, val in zip(bars, rates):
    ax.text(bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.3,
            f"{val:.1f}%", ha="center", va="bottom",
            fontsize=11, fontweight="bold")

ax.set_title("Phishing Rate (%) by URL Length Bin", fontsize=14, fontweight="bold")
ax.set_xlabel("URL Length Bin", fontsize=11)
ax.set_ylabel("Phishing Rate (%)", fontsize=11)
ax.set_ylim(0, rates.max() * 1.25)
ax.grid(axis="y", alpha=0.3, linestyle="--")

patches = [
    mpatches.Patch(color="#4CAF50", label="Low risk (<15%)"),
    mpatches.Patch(color="#FF9800", label="Medium risk (15–22%)"),
    mpatches.Patch(color="#E53935", label="High risk (>22%)"),
]
ax.legend(handles=patches, fontsize=9, loc="upper right")

plt.tight_layout()
plt.savefig("plot4_phishing_rate.png", dpi=150, bbox_inches="tight")
plt.show()
```

Stacked Bar Chart:

```
# Plot 5: Stacked Bar Chart — Volume Composition per Bin
fig, ax = plt.subplots(figsize=(10, 5))

bin_counts = df.groupby(["LengthBin", "Label"], observed=True).size().unstack(fill_value=0)
bin_counts[["Legitimate", "Phishing"]].plot(
    kind="bar", stacked=True, ax=ax,
    color=["#2196F3", "#E53935"], edgecolor="white", linewidth=0.5, width=0.6
)

ax.set_title("URL Count by Length Bin — Legitimate vs. Phishing (Stacked)",
             fontsize=13, fontweight="bold")
ax.set_xlabel("URL Length Bin", fontsize=11)
ax.set_ylabel("Number of URLs", fontsize=11)
ax.set_xticklabels(ax.get_xticklabels(), rotation=20, ha="right")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{int(x):,}"))
ax.legend(title="Class", fontsize=10)
ax.grid(axis="y", alpha=0.3, linestyle="--")

plt.tight_layout()
plt.savefig("plot5_stacked_bar.png", dpi=150, bbox_inches="tight")
plt.show()
```

Interpretation:

```
# Key Findings & Interpretation:

top_bin  = bin_stats["Phishing_Rate_%"].idxmax()
top_rate = bin_stats["Phishing_Rate_%"].max()

print("=" * 60)
print("         URL LENGTH vs. PHISHING — KEY FINDINGS")
print("=" * 60)
print(f"  Total URLs analyzed     : {len(df):,}")
print(f"  Legitimate              : {(df.Phising==0).sum():,} ({(df.Phising==0).mean()*100:.1f}%)")
print(f"  Phishing                : {(df.Phising==1).sum():,} ({(df.Phising==1).mean()*100:.1f}%)")
print()
print(f"  Avg length — Legitimate : {legit.mean():.1f} chars")
print(f"  Avg length — Phishing   : {phish.mean():.1f} chars")
print(f"  Difference              : {legit.mean()-phish.mean():.1f} chars (legit URLs are LONGER)")
print()
print(f"  Pearson correlation     : {df['UrlLength'].corr(df['Phising']):.4f} (negative = longer -> less phishing)")
print(f"  Highest-risk bin        : {top_bin} -> {top_rate:.1f}% phishing rate")
print()
print("  INTERPRETATION:")
print("  Contrary to common assumptions, phishing URLs in this")
print("  dataset tend to be SHORTER than legitimate ones. Very")
print("  short URLs (≤50 chars) have a ~22% phishing rate, and")
print("  very long URLs (500+) spike again at ~26%. The safest")
print("  range is 51–200 characters (~8–9% phishing rate).")
print("=" * 60)
```